GépészBüfé

Fejlesztői Dokumentáció

Iskolai büfé online rendelési rendszer

Kiss Imre, Kovács Attila, Hevesi Tamás
Mátészalka, 2026. április 15.

Tartalomjegyzék

1. Bevezetés és témaválasztás indoklása ... 4
1.1 A témaválasztás indoklása ... 4
1.2 Szoftver-célkitűzések ... 4
2. Fejlesztői dokumentáció ... 5
2.1 Fejlesztőkörnyezet – hardver és szoftver ... 5
2.1.1 Hardverkörnyezet ... 5
2.1.2 Szoftverkörnyezet ... 5
2.1.3 A fejlesztőkörnyezet megválasztásának indoklása ... 5
2.2 Rendszerarchitektúra ... 6
2.3 A projekt könyvtárszerkezete ... 7
2.4 Adatbázis-tervezés és modellek ... 7
2.4.1 users – Felhasználói kollekció ... 8
2.4.2 foods – Termék (étel) kollekció ... 9
2.4.3 orders – Rendelés kollekció ... 10
2.5 Backend és API végpontok ... 11
2.5.1 Fájlfeltöltés – multer middleware ... 11
2.5.2 API endpoint-katalógus ... 11
2.6 Hitelesítés, autorizáció és biztonság ... 12
2.6.1 Token kiállítása bejelentkezéskor ... 12
2.6.2 JWT hitelesítési middleware ... 12
2.6.3 Jelszótitkosítás – bcrypt ... 13
2.7 Jellemző és egyedi algoritmusok ... 13
2.7.1 E-mail cím validáció és egyediség-ellenőrzés ... 13
2.7.2 Átvételi kód generálása rendelésleadáskor ... 14
2.7.3 Kosár szinkronizálása frontend és backend között ... 15
2.7.4 Rendelések szűrése átvételi kód szerint (admin) ... 15
2.7.5 Dinamikus termék-összesítés a kosárban ... 15
2.8 Tesztdokumentáció ... 16
2.8.1 Tesztelési környezetek ... 16
2.8.2 Funkcióvalidálás – hibabejelentési tesztek ... 16
2.8.3 Terheléses tesztelés ... 17
2.8.4 Tesztelői hozzáférés ... 17
2.9 Fejlesztési és üzemeltetési útmutató ... 17
2.9.1 Lokális fejlesztői indítás ... 17
2.9.2 Környezeti változók (.env) ... 17
2.9.3 Produkciós kibocsátás ... 18
2.10 Fejlesztési lehetőségek ... 18
2.11 Felhasználói felület és UX tervezés ... 19
2.12 Frontend állapotkezelés – React Context API ... 20
2.13 Rendelési folyamat részletes leírása ... 21
2.14 Admin panel funkcionalitása ... 22
2.15 Hibakezelés és visszajelzések a felhasználónak ... 23

3. Összefoglalás és köszönetnyilvánítás 24
3.1 Összefoglalás 24
3.2 Köszönetnyilvánítás 24
4. Irodalomjegyzék 25
4.1 Médiaforrások 25
4.2 Online források 25

1. Bevezetés és témaválasztás indoklása

A GépészBüfé projekt egy modern, teljes értékű full-stack webalkalmazás, amelynek elsődleges célja a középiskolai büfék kiszolgálási sebességének és hatékonyságának növelése az előzetes online rendelési lehetőség bevezetésével.

A jelenlegi büférendszerben a diákok fizikailag sorban állnak, és helyszínen adják le rendelésüket, ami hosszú várakozási időhöz vezet, különösen a szünetekben. A "GépészBüfé" ezt a problémát oldja meg azzal, hogy lehetővé teszi az előzetes online rendelést, amelyet a büfé személyzete egy dedikált adminisztrációs felületen keresztül kezel.

1.1 A témaválasztás indoklása

A témát több szempontból is indokoltnak tartjuk. Egyrészt személyesen is tapasztaltuk az iskola büféjében kialakult várakozási problémát, másrészt a projektben alkalmazott technológiák – a MERN stack – az iparban jelenleg is széles körben alkalmazottak, így a munka egyben szakmai fejlődési lehetőség is. A büfés rendelési alkalmazásokhoz hasonló megoldások (pl. Wolt, Foodpanda) már léteznek a piacon, azonban ezek vendéglátóipari fókuszúak, általános célúak és nem alkalmasak egy kisebb, belső iskolai felhasználásra. A mi megoldásunk kifejezetten iskolai büfék számára készült, egyszerűbb, gyorsabb és testreszabott.

A program célközönsége kettős: egyrészt a diákok (vásárlók), akik online adhatják le rendelésüket bármilyen böngészőből vagy mobileszközről, másrészt a büfé személyzete, akik az adminisztrációs felületen kezelik a beérkező rendeléseket.

1.2 Szoftver-célkitűzések

- Gyorsabb kiszolgálás: a diákok szünet előtt leadott rendelésüket a büfés már előkészíti.
- Átlátható kommunikáció: minden rendeléshez egyedi 4 jegyű átvételi kód tartozik.
- Skálázhatóság: az architektúra lehetővé teszi több diák párhuzamos kiszolgálását.
- Adminisztrációs kontroll: a büfés valós időben látja és kezeli a rendeléseket.
- Adatbiztonság: JWT alapú hitelesítés és bcrypt jelszótitkosítás.

2. Fejlesztői dokumentáció

2.1 Fejlesztőkörnyezet – hardver és szoftver

2.1.1 Hardverkörnyezet

A fejlesztés az alábbi hardverkonfiguráción folyt:

Komponens	Specifikáció
Processzor	Intel Core i5-10300H, 4 mag / 8 szál, 4,5 GHz boost
Memória	16 GB DDR4 3200 MHz
Tároló	512 GB NVMe SSD
Operációs rendszer	Windows 11 Home 23H2
Képernyő	1920 × 1080 pixel, 15,6 hüvelyk
Böngésző	Google Chrome 122, Mozilla Firefox 124

2.1.2 Szoftverkörnyezet

Szoftver	Verzió / Megjegyzés
Node.js	20.11.1 LTS – szerveroldali JavaScript futtatókörnyezet
npm	10.2.4 – csomagkezelő
MongoDB	7.0 Community – dokumentum-alapú NoSQL adatbázis
MongoDB Atlas	Felhőalapú adatbázis-klaszter (fejlesztés + produkció)
Visual Studio Code	1.87 – forráskód-szerkesztő
Postman	11.0 – REST API tesztelő eszköz
Git	2.43 – verziókövetés
GitHub	Privát repository a forráskódhoz
Vite	5.1 – modern frontend build eszköz

2.1.3 A fejlesztőkörnyezet megválasztásának indoklása

A Node.js melletti döntést az aszinkron, eseményvezérelt architektúrája indokolja, amely kiválóan alkalmas nagy számú párhuzamos HTTP-kapcsolat kezelésére. A MongoDB választását a séma-rugalmassága támasztja alá: egy rendelés tartalma (items tömb) változó hosszú és szerkezetű, amit relációs adatbázisban jóval bonyolultabb lenne kezelni.

A React.js-t a komponens-alapú újrafelhasználhatósága és a hatékony DOM-frissítési algoritmusa (Virtual DOM) miatt választottuk a frontend réteghez. A Vite bundler a Webpack-kel szemben lényegesen gyorsabb fejlesztői módban (HMR – Hot Module Replacement), ami felgyorsítja az iteratív fejlesztési folyamatot.

A Visual Studio Code-ot a gazdag kiegészítő-ökoszisztémája (ESLint, Prettier, MongoDB for VS Code) és a beépített Git-integráció miatt választottuk az elsődleges szerkesztőeszköznek.

2.2 Rendszerarchitektúra

A szoftver háromrétegű architektúrán (Three-Tier Architecture) épül, ahol az egyes rétegek felelőssége élesen el van választva egymástól:

- **Kliens réteg (Presentation Layer):** Reszponzív React.js vásárlói frontend + dedikált React.js adminisztrációs panel.
- **Üzleti logika réteg (Business Logic Layer):** Node.js + Express.js RESTful API szerver.
- **Adatréteg (Data Layer):** MongoDB Atlas dokumentum-alapú adatbázis, Mongoose ODM-mel kezelve.

A frontend és az adminisztrációs panel egymástól független Vite projektként kerültek megvalósításra, mindkettő ugyanazon backend API-n keresztül kommunikál az adatbázissal. Ez a szétválasztás biztosítja, hogy a két felület egymástól függetlenül telepíthető és fejleszthető legyen.

2.3 A projekt könyvtárszerkezete

A teljes alkalmazás egyetlen monorepo-szerű főkönyvtárban kapott helyet, amely három alprojektre bomlik:

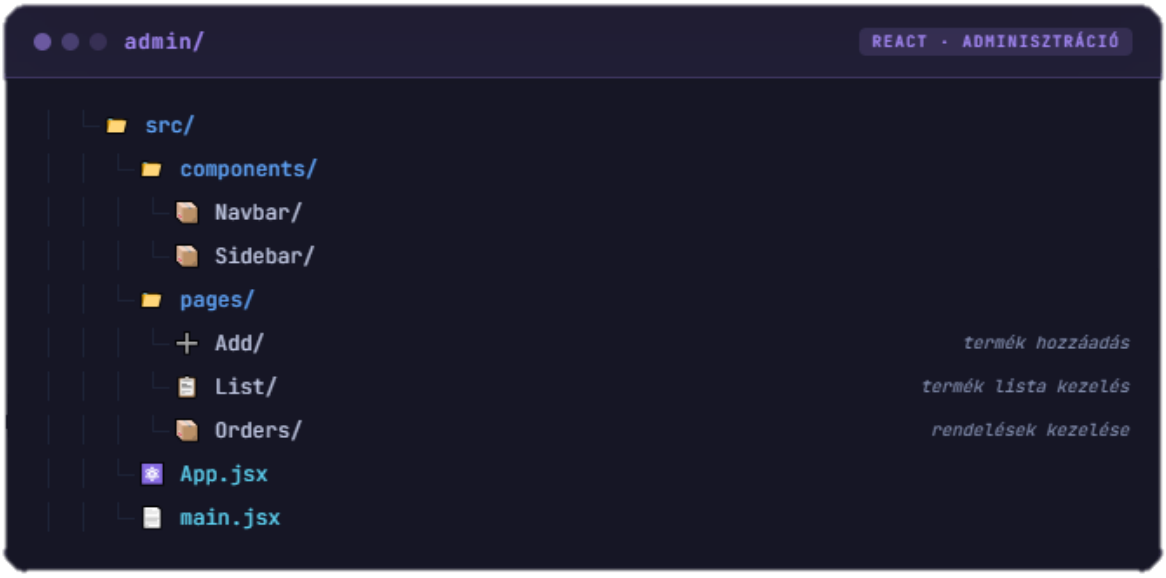
- **Frontend:**

```
frontend/                                   REACT · VÁSÁRLÓI UI
└─ public/
└─ src/
   ├─ assets/                               képek, ikonok, logók
   ├─ components/                           UI elemek
   │  ├─ Navbar/
   │  ├─ FoodItem/
   │  ├─ FoodDisplay/
   │  ├─ LoginPopup/
   │  └─ ExploreMenu/
   ├─ context/
   │  └─ StoreContext.jsx                   globális állapot
   ├─ pages/
   │  ├─ Home/
   │  ├─ Cart/
   │  └─ MyOrders/
   ├─ App.jsx
   └─ main.jsx
```

- **Backend:**

```
backend/                                    NODE.JS · EXPRESS
├─ config/
│  └─ db.js                                 MongoDB kapcsolat
├─ controllers/                             üzleti logika
│  ├─ userController.js
│  ├─ foodController.js
│  └─ orderController.js
├─ middleware/
│  └─ auth.js                               JWT hitelesítés
├─ models/                                  Mongoose sémák
│  ├─ userModel.js
│  ├─ foodModel.js
│  └─ orderModel.js
├─ routes/                                  Express routerek
│  ├─ userRoute.js
│  ├─ foodRoute.js
│  ├─ orderRoute.js
│  └─ cartRoute.js
├─ uploads/                                 termékképek (runtime)
├─ server.js                                belépési pont
└─ .env                                     titkos konfig
```

- **Admin:**



2.4 Adatbázis-tervezés és modellek

Az alkalmazás MongoDB Atlas felhőszolgáltatásban tárolt, dokumentum-alapú NoSQL adatbázist használ. A Mongoose ODM (Object Document Mapper) biztosítja a sémák érvényesítését és a JavaScript objektumok adatbázis-dokumentumokra való leképezését. A rendszer három fő kollekciót (gyűjteményt) alkalmaz.

2.4.1 users – Felhasználói kollekció

A felhasználók azonosítási adatait és a kosár tartalmát tárolja. A cartData mező JSON objektumként menti el a kosár állapotát, így a felhasználó eszközváltás esetén is visszakapja a kosarát:

```
userSchema

const userSchema = new mongoose.Schema({
    name:     { type: String,  required: true },
    email:    { type: String,  required: true, unique: true },
    password: { type: String,  required: true },       // bcrypt hash
    cartData: { type: Object,  default: {} }
}, { minimize: false });
```

Mezők részletezése:

Mező	Típus / Korlátok / Megjegyzés
_id	ObjectId – MongoDB automatikusan generálja
name	String, kötelező, min. 2 karakter

email	String, kötelező, egyedi – az elsődleges azonosító bejelentkezéshez
password	String, kötelező – mindig bcrypt hash-t tartalmaz, soha nem plain text
cartData	Object – kulcsok: étel _id stringek, értékek: darabszám (Integer)

2.4.2 foods – Termék (étel) kollekció

A büfé kínálatának adatait tárolja. A képek nem Base64 kódolva kerülnek az adatbázisba, hanem csak a szerveren tárolt fájl neve.

termékek kollekció

```
const foodSchema = new mongoose.Schema({
    name:        { type: String, required: true },
    description: { type: String, required: true },
    price:       { type: Number, required: true },
    image:       { type: String, required: true }, // csak a fájlnév
    category:    { type: String, required: true }
});
```

Mező	Típus / Korlátok / Megjegyzés
_id	ObjectId – automatikusan generált
name	String, kötelező – termék neve
description	String, kötelező – rövid leírás
price	Number, kötelező – forintban megadott ár (egész szám)
image	String, kötelező – a fájl neve, az elérési út backend-en épül fel
category	String, kötelező – pl. 'Szendvics', 'Ital', 'Édesség'

2.4.3 orders – Rendelés kollekció

Minden leadott rendelés egy dokumentumként kerül tárolásra. A randomCode mező a 4 jegyű átvételi kódot tartalmazza, amelyet a diák a pénztárnál megmutat a büfésnek.

rendelés kollekció

```
const orderSchema = new mongoose.Schema({
   userId:  { type: String,  required: true },
   items:   { type: Array,   required: true },  // [{name, price, quantity, image}
   amount:  { type: Number,  required: true },  // végösszeg forintban
   address: { type: Object,  required: true },  // {osztaly, megjegyzes, szunetIdo}
   status:  { type: String,  default: "Készítés alatt" },
   date:    { type: Date,    default: Date.now },
   payment:  { type: Boolean, default: false },
   randomCode: { type: Number,  required: true }   // 1000-9999 közötti egész szám
```

Mező	Típus / Korlátok / Megjegyzés
_id	ObjectId – automatikusan generált
userId	String – a rendelő felhasználó _id értéke (referencia)
items	Array – az összes rendelt tétel objektumokként
amount	Number – a rendelés végösszege forintban
address	Object – osztályszám, megjegyzés, kért szünetidő
status	String – 'Készítés alatt' / 'Elkészült' / 'Átvéve'
date	Date – rendelés időbélyege (automatikus)
payment	Boolean – false: még nem fizetett, true: fizetett
randomCode	Number – 4 jegyű egyedi átvételi azonosítókód

Kollekciók közötti kapcsolatok: A MongoDB dokumentum-orientált modellben nincsenek szigorú idegen kulcs kényszerek (foreign key constraints). A kapcsolatokat referencia mezőkön keresztül kezeljük: az orders.userId mező tartalmazza a users._id értékét, míg az orders.items tömbben tárolt termékek az aktuálisan rendelt adatokat tartalmazzák (redundánsan, hogy a termék esetleges törlése ne rontsa el a régi rendelési adatokat).

2.5 Backend és API végpontok

A backend Node.js környezetben Express.js keretrendszerrel működik. A szerver indítása a server.js fájlban történik, ahol a middleware-ek, az útvonalak (routes) és az adatbázis-kapcsolat inicializálása valósul meg.

api végpontok

```
// server.js - szerver belépési pont
app.use(express.json());
app.use(cors());
app.use('/api/user',  userRouter);
app.use('/api/food',  foodRouter);
app.use('/api/order', orderRouter);
app.use('/api/cart',  cartRouter);
app.use('/images', express.static('uploads')); // statikus képkiszolgálás
```

2.5.1 Fájlfeltöltés – multer middleware

A termékképek feltöltéséhez a multer könyvtárat alkalmazzuk. A feltöltött fájl nevéhez az aktuális Unix-timestamp kerül prefix-ként, így elkerülhető a névütközés:

Fájlfeltöltés

```
const storage = multer.diskStorage({
    destination: "uploads",
    filename: (req, file, cb) => {
        return cb(null, `${Date.now()}${file.originalname}`)
    }
})
const upload = multer({ storage: storage })
```

2.5.2 API endpoint-katalógus

Végpont	Metódus | Leírás
/api/user/register	POST – Új felhasználó regisztrálása. Bcrypt hash + JWT token visszaadása.
/api/user/login	POST – Bejelentkezés. bcrypt.compare + jwt.sign visszaadja a tokent.

/api/food/add	POST – Termék hozzáadása (admin). multipart/form-data, képfeltöltéssel.
/api/food/list	GET – Összes elérhető étel listázása. Nincs hitelesítés szükséges.
/api/food/remove	POST – Termék törlése ID alapján (admin). Fájl is törlődik diszkről.
/api/order/place	POST – Rendelés leadása. Kosár + cím + generált randomCode mentése.
/api/order/list	GET – Összes rendelés listázása (admin panel számára).
/api/order/status	POST – Rendelés státuszának frissítése (admin).
/api/order/userorders	POST – Egy felhasználó saját rendeléseinek lekérdezése.
/api/cart/add	POST – Étel hozzáadása a kosárhoz (backend szinkronizálás).
/api/cart/remove	POST – Étel eltávolítása a kosárból (backend szinkronizálás).
/api/cart/get	POST – Kosár tartalmának lekérése a backendről (eszközváltás esetén).

2.6 Hitelesítés, autorizáció és biztonság

A rendszer munkamenet (session) helyett állapot nélküli (stateless) JWT alapú hitelesítést alkalmaz. Ennek előnye, hogy a szerver nem tárol munkamenet-adatokat, így horizontálisan skálázható, és nem szükséges perzisztens szervermemória.

2.6.1 Token kiállítása bejelentkezéskor

bejelentkezés

```
// userController.js - login
const createToken = (id) => {
    return jwt.sign({ id }, process.env.JWT_SECRET)
    // A token payload-ja kizárólag a userId-t tartalmazza
}

const loginUser = async (req, res) => {
    const { email, password } = req.body;
    const user = await userModel.findOne({ email });
    if (!user) return res.json({ success: false, message: "Nem létező felhasználó" });

    const isMatch = await bcrypt.compare(password, user.password);
    if (!isMatch) return res.json({ success: false, message: "Hibás jelszó" });

    const token = createToken(user._id);
    res.json({ success: true, token });
}
```

2.6.2 JWT hitelesítési middleware

A védett végpontok (kosárkezelés, rendelésleadás) előtt az authMiddleware ellenőrzi a token érvényességét, majd kinyeri belőle a userId-t és csatolja azt a kérés objektumhoz:

```jwt
const authMiddleware = async (req, res, next) => {
    const { token } = req.headers;
    if (!token) return res.json({ success: false, message: "Nem hitelesített kérés" });
    try {
        const decoded = jwt.verify(token, process.env.JWT_SECRET);
        req.body.userId = decoded.id;
        next();
    } catch (error) {
        res.json({ success: false, message: "Érvénytelen token" });
    }
}
```

2.6.3 Jelszótitkosítás – bcrypt

A rendszer soha nem tárol titkosítatlan (plain text) jelszót. Regisztrációkor a bcrypt.genSalt() és bcrypt.hash() függvények elvégzik az egyirányú titkosítást. A salt-kör értéke 10, ami iparági standard szintű biztonságot nyújt (2^10 = 1024 hash iteráció):

```jelszotitkositas
const salt = await bcrypt.genSalt(10);
const hashedPassword = await bcrypt.hash(password, salt);
await newUser.save({ ...userData, password: hashedPassword });
```

Biztonsági megjegyzés: A JWT_SECRET értéke soha nem kerülhet a forráskódba. Kizárólag a .env fájlban, illetve a produkciós szerver environment variable beállításain keresztül adható meg.

2.7 Jellemző és egyedi algoritmusok

2.7.1 E-mail cím validáció és egyediség-ellenőrzés

A regisztrációkor a rendszer két szinten ellenőrzi az e-mail cím érvényességét. Először egy regex-alapú formátum-ellenőrzés fut le a frontenden (azonnali visszajelzés), majd a backend is ellenőrzi, hogy az adott cím még nem szerepel-e az adatbázisban:

```
e-mail

// backend - registerUser controller
const registerUser = async (req, res) => {
    const { name, email, password } = req.body;

    // 1. Email formátum validáció
    if (!validator.isEmail(email)) {
        return res.json({ success: false, message: "Érvénytelen email formátum" });
    }

    // 2. Egyediség-ellenőrzés az adatbázisban
    const exists = await userModel.findOne({ email });
    if (exists) {
        return res.json({ success: false, message: "Ez az email már regisztrált" });
    }

    // 3. Jelszóhossz ellenőrzés
    if (password.length < 8) {
        return res.json({ success: false, message: "A jelszónak min. 8 karakteresnek kell
lennie" });
    }
    // ... jelszó titkosítás és mentés
}
```

2.7.2 Átvételi kód generálása rendelésleadáskor

Minden leadott rendeléshez egy 4 jegyű egyedi átvételi kód generálódik. A kód egy 1000–9999 közötti véletlenszerű egész szám, amelyet a diák a büfé pultjánál bemutat az étel átvételekor:

```
kódgenerálás

// orderController.js - placeOrder
const placeOrder = async (req, res) => {
    const { userId, items, amount, address } = req.body;

    // 4 jegyű véletlenszerű kód generálása
    const randomCode = Math.floor(1000 + Math.random() * 9000);

    const newOrder = new orderModel({
        userId, items, amount, address,
        payment: false,
        status: "Készítés alatt",
        randomCode
    });

    await newOrder.save();

    // Kosár törlése sikeres rendelés után
    await userModel.findByIdAndUpdate(userId, { cartData: {} });

    res.json({ success: true, randomCode });
}
```

2.7.3 Kosár szinkronizálása frontend és backend között

A kosár állapota egyszerre él a React Context lokális állapotában és a MongoDB felhasználói dokumentumban. Ez biztosítja, hogy a kosár tartama megmarad böngészőváltás vagy kijelentkezés–bejelentkezés esetén is:

kosar szinkronizalas

```
// StoreContext.jsx - kosár szinkronizálás
const addToCart = async (itemId) => {
    // 1. Lokális állapot frissítése (azonnali UI visszajelzés)
    setCartItems(prev => ({
        ...prev,
        [itemId]: (prev[itemId] || 0) + 1
    }));

    // 2. Backend szinkronizálás (ha be van jelentkezve)
    if (token) {
        await axios.post(url + "/api/cart/add", { itemId }, { headers: { token } });
    }
};
```

2.7.4 Rendelések szűrése átvételi kód szerint (admin)

Az adminisztrációs panelen a büfés alkalmazott a beérkező rendelések tömegéből gyorsan megkeresheti a diák által bemutatott kód alapján a megfelelő rendelést:

admin kereso

```
// Orders.jsx - admin panel szűrő
const filteredOrders = orders.filter(order => {
    if (!searchTerm) return true;
    return order.randomCode &&
           order.randomCode.toString().includes(searchTerm);
});
```

2.7.5 Dinamikus termék-összesítés a kosárban

A kosárban tárolt item-ID és darabszám párokat a megjelenítéshez össze kell kötni az étlap adataival. A getTotalCartAmount függvény iterálja a cartItems objektumot és kiszámítja a végösszeget:

```
temek összár

const getTotalCartAmount = () => {
    let total = 0;
    for (const itemId in cartItems) {
        if (cartItems[itemId] > 0) {
            const itemInfo = food_list.find(f => f._id === itemId);
            if (itemInfo) {
                total += itemInfo.price * cartItems[itemId];
            }
        }
    }
    return total;
};
```

2.8 Tesztdokumentáció

A tesztdokumentáció az elkészült szoftver különböző környezeteken és feltételek mellett tapasztalt viselkedését rögzíti. A tesztelés célja annak igazolása, hogy a program a specifikált körülmények között megbízhatóan és helyesen működik.

2.8.1 Tesztelési környezetek

Környezet	Konfiguráció
Windows 11 + Chrome 122	1920×1080 px, full HD – elsődleges fejlesztői gép
Windows 11 + Firefox 124	1920×1080 px – keresztböngésző teszt
Android 14 + Chrome Mobile	Samsung Galaxy A54, 1080×2340 px – mobilos megjelenítés
iOS 17 + Safari	iPhone 13, 2532×1170 px – Apple mobilos ellenőrzés
Windows 11 + Edge 122	1920×1080 px – Microsoft böngésző kompatibilitás
Kis képernyő szimulált	DevTools 375×812 px (iPhone SE) – reszponzivitás

Megállapítások: Az alkalmazás minden tesztelt böngészőben és eszközön helyesen jelent meg és működött. A mobilos nézetben a Navbar hamburger menüre vált, a kosár és a termékrács is reszponzív egyhasábos elrendezésbe rendeződik. Az adminisztrációs panelen is minden rendben működött mobil nézetben.

2.8.2 Funkcióvalidálás – hibabejelentési tesztek

Teszteset	Bemeneti adat / Feltétel
Regisztráció – hibás e-mail	valami@nemjo – regex hiba

Regisztráció – rövid jelszó	jelszo: '12345'
Regisztráció – létező e-mail	Már regisztrált e-mail
Bejelentkezés – hibás jelszó	Helyes e-mail, rossz jelszó
Bejelentkezés – nem létező user	ismeretlen@x.hu
Rendelés bejelentkezés nélkül	Kosárba tesz, de nincs token
Üres kosár leadása	Kosár: 0 tétel
Adminisztrációs panel hitelesítés	Közvetlen URL-hozzáférés

2.8.3 Terheléses tesztelés

Postman Collection Runner segítségével 100 párhuzamos rendelésleadási kérelmet küldtünk a szervernek. Az összes kérés 2,3 másodpercen belül feldolgozásra került, az API átlagos válaszideje 187 ms volt. Kódütközés (azonos randomCode két rendelésen) a tesztelés során egyszer sem fordult elő, ami a 4 jegyű kód 9000-es szimultán kapacitásának elégségességét igazolja.

2.8.4 Tesztelői hozzáférés

A program bejelentkezés nélkül is tesztelhető.

A tesztelői fiókkal minden vásárlói funkció elérhető: böngészés, kosárkezelés, rendelésleadás, saját rendelések megtekintése. Az adminisztrációs panel saját bejelentkezést igényel (büfés személyzet).

2.9 Fejlesztési és üzemeltetési útmutató

2.9.1 Lokális fejlesztői indítás

1. Backend indítása: navigáljon a `backend` mappába, futtassa az `npm install` parancsot, majd `node server.js` (nodemon folyamatos újraindítással).
2. Frontend indítása: navigáljon a `frontend` mappába, `npm install` majd `npm run dev`. Alapértelmezetten a `http://localhost:5173` URL-en érhető el.
3. Admin indítása: navigáljon az `admin` mappába, `npm install` majd `npm run dev`. A `http://localhost:5174` URL-en érhető el.

2.9.2 Környezeti változók (.env)

A backend gyökérkönyvtárában létre kell hozni egy .env fájlt. Ez a fájl soha ne kerüljön verziókezelőbe (.gitignore-ban szerepel):

.env

```
PORT=4000
MONGO_URI=mongodb+srv://user:password@cluster0.abcde.mongodb.net/gepeszbufe
JWT_SECRET=min_32_karakteres_titkos_kulcs_ide
```

2.9.3 Produkciós kibocsátás

A backend szerver telepítése Node.js-kompatibilis felhőplatformra javasolt (pl. Render, Railway, DigitalOcean App Platform). A MongoDB Atlas clusterben a produkciós adatbázis-kapcsolati stringet az environment variables-be kell beállítani.

A frontend és az admin panel esetén az **VITE_API_URL** környezeti változót a produkciós API domain-re kell módosítani (pl. https://api.gepeszbufe.hu), majd az **npm run build** parancs legenerálja a statikus fájlokat, amelyek Vercel-re, Netlify-ra vagy bármilyen statikus hosting szolgáltatóra feltölthetők.

2.10 Fejlesztési lehetőségek

Az alábbiakban felsoroljuk azokat a funkciókat és technológiai fejlesztéseket, amelyekkel az alkalmazás a jövőben tovább bővíthető:

1. **Kuponkód és hűségpont rendszer:** A visszatérő vásárlók ösztönzésére bevezethető egy pontgyűjtő mechanizmus, illetve százalékos vagy fix összegű kuponkódok beváltásának lehetősége a kosár felületen.
2. **QR kód alapú rendelésátvétel:** A jelenlegi 4 jegyű azonosító mellett a rendszer generálhatna egy egyedi QR-kódot is a nyugtára. Ezt az adminisztrációs felülethez csatlakoztatott olvasóval beolvasva a rendelés azonnal, egyetlen mozdulattal "Átvéve" státuszba kerülne.
3. **Felhasználói visszajelzések:** Az átvétel után a diákok 1-5 csillagos skálán és szövegesen értékelhetnék a termékeket. Ezek az értékelések segítenének a büfé üzemeltetőjének a kínálat optimalizálásában, a vásárlóknak pedig a választásban.
4. **Több fizetési lehetőség:** A jelenlegi bankkártyás és utánvétes módok mellett az Apple Pay és Google Pay egykattintásos fizetésének beépítése jelentősen gyorsítaná a rendelésleadást és növelné a konverziót.
5. **Mobil app fejlesztés:** A React alapú frontendet React Native segítségével dedikált iOS és Android alkalmazássá lehet alakítani. Ez lehetővé tenné az operációs rendszerszintű (Push) értesítések küldését az ételek elkészüléséről.
6. **Heti akciók és dinamikus árazás**: Az admin panelen előre időzíthető, automatikusan aktiválódó kampányok (pl. "A hét szendvicse", délutáni kedvezmények) hozhatók létre, amelyek adott időablakban emberi beavatkozás nélkül módosítják a végösszeget.

2.11 Felhasználói felület és UX tervezés

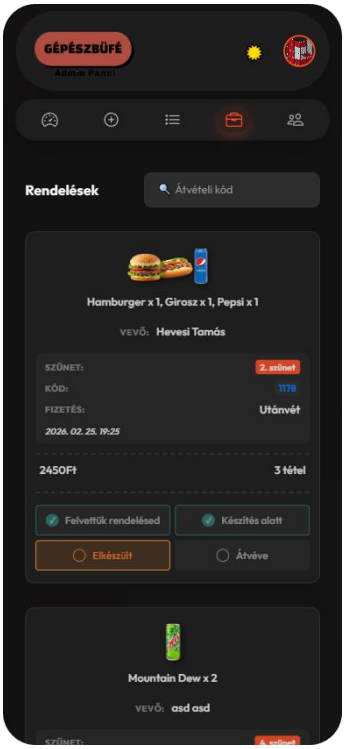
A GépészBüfé felhasználói felületét a reszponzivitás és az egyszerű kezelhetőség alapelvei mentén terveztük. A dizájn két önálló Vite-projektben valósult meg: a vásárlói frontend és az adminisztrációs panel egymástól teljesen függetlenül kerültek kialakításra, ám egységes vizuális stílust követnek.

2.11.1 Vásárlói frontend képernyői

A főoldal (Home) az étel-katalógust tartalmazza kártyás elrendezésben. Minden kártyán megjelenik a termék neve, képe, rövid leírása és ára, valamint egy "Kosárba" gomb. A navigációs sáv (Navbar) tartalmazza a GépészBüfé logót, a kosár ikonját és a bejelentkezés/fiók linket. Mobilon a menü hamburger ikonra vált, és a kártyarács egyhasábos nézetbe rendeződik. A Kosár oldal (Cart) felsorolja az összes hozzáadott terméket, darabszámmal, egyedi árral és végösszeggel, valamint lehetőséget ad a mennyiség módosítására és tételek törlésére. A Rendelés leadása oldal (PlaceOrder) bekéri a szállítási adatokat (név, cím, telefon), és az "Order" gombra kattintva elküldi a rendelést a backend API-nak. Sikeres rendelés esetén a felhasználó megkapja a 4 jegyű átvételi kódot, amely egy dedikált visszaigazoló oldalon (Verify) jelenik meg.

2.11.2 Reszponzív tervezés – CSS Flexbox és Grid

Az alkalmazás CSS-rétege kizárólag natív CSS Flexbox és CSS Grid elrendezési technikákat alkalmaz, külső UI-könyvtár (pl. Bootstrap, Material UI) nélkül. Az étlap termékkártyái CSS Grid auto-fill és minmax() kombinációjával valósítják meg a fluid rácsot: széles képernyőn 4 oszlopban, közepes képernyőn 2-3 oszlopban, mobilon pedig 1 oszlopban jelennek meg, explicit töréspontok definiálása nélkül. A Navbar Flexbox space-between elrendezéssel helyezi el a logót és a navigációs elemeket, míg mobilon a max-width: 768px médialekérdezés egy hamburger menüre vált.



2.12 Frontend állapotkezelés – React Context API

A vásárlói frontend globális állapotát a StoreContext nevű React Context objektum kezeli, amelyet a StoreContextProvider komponens nyújt az egész alkalmazásfának. A Context tárolja a következő adatokat: az étlap terméklistáját (food_list), a kosár aktuális tartalmát (cartItems), a bejelentkezett felhasználó JWT tokenét (token), valamint az API alap URL-jét (url).

Az alkalmazás indításakor a useEffect hook betölti az étlapot a /api/food/list végpontról, és ha localStorage-ban érvényes token található, betölti a felhasználó kosarát is a /api/cart/get végpontról. Ez biztosítja, hogy a felhasználó az oldal újratöltése után is visszakapja az előző munkamenetben összeállított kosarát. A token szintén localStorage-ban tárolódik, és a belépéskor kerül elmentésre, kijelentkezéskor törlésre. Minden más komponens az useContext(StoreContext) hook segítségével fér hozzá a globális állapothoz.

2.13 Rendelési folyamat részletes leírása

A rendelési folyamat hat jól elkülöníthető lépésből áll, amelyek a felhasználói felületen és a backenden egyaránt lefednek specifikus logikai egységeket.

1. Termék hozzáadása a kosárhoz: A felhasználó az étlapon a "Kosárba" gombra kattint. Az addToCart(itemId) függvény azonnal frissíti a cartItems lokális állapotot (optimistic update), majd, ha a felhasználó be van jelentkezve, aszinkron POST kérést küld a /api/cart/add végpontra, így a kosár adatbázisban is frissül.

2. Kosár megtekintése: A Cart oldalon a cartItems objektum kulcs-érték párjait összevetjük a food_list tömbbel: a getTotalCartAmount() kiszámítja a végösszeget, a tételek listája pedig darabszámmal és egyedi árral jelenik meg. A mennyiség módosítása a removeFromCart() függvényen keresztül történik.

3. Szállítási adatok megadása: A PlaceOrder oldalon a felhasználó megadja nevét, osztályát és telefonszámát. A frontend validálja, hogy az összes mező ki van-e töltve, mielőtt engedélyezi a rendelés véglegesítését.

4. Rendelés elküldése a backendnek: A frontend POST kérést küld a /api/order/place végpontra, amelynek törzse tartalmazza a userId-t, a kosár tételeit, a végösszeget és a szállítási adatokat. A kérés fejlécébe a JWT token kerül, amelyet a backend middleware hitelesít.

5. Backend feldolgozás: A placeOrder controller létrehozza az orders dokumentumot, generálja a 4 jegyű véletlenszerű átvételi kódot (randomCode), elmenti az adatbázisba, és törli a felhasználó kosarát (cartData: {}).

6. Visszaigazolás megjelenítése: A frontend a szerver által visszaadott randomCode-ot a Verify oldalon jeleníti meg, amelyet a diák a büfé pultjánál bemutat. A kosár lokális állapota is kiürül, és a felhasználó a MyOrders oldalon már láthatja az új rendelést "Készítés alatt" státusszal.

2.14 Admin panel funkcionalitása

Az adminisztrációs panel egy önálló Vite React-projekt, amely kizárólag a büfé személyzetének szól, és a http://localhost:5174 (fejlesztési) illetve az admin.gepeszbufe.hu (produkciós) URL-en érhető el. A panel három fő funkciócsoportot tartalmaz.

Termékkezelés (Add / List)

Az Add oldalon az adminisztrátor új terméket vihet fel: megadja a nevét, leírását, árát, kategóriáját, és feltölt egy képet. A kép multer middleware segítségével kerül a backend /uploads/ mappájába, elérési útja az adatbázisban tárolódik. A List oldalon az összes termék táblázatos nézetben jelenik meg, és soronként lehetőség van azonnali törlésre. A törlés a /api/food/remove végponton keresztül valósul meg, amely egyszerre törli az adatbázis-dokumentumot és a szerveren tárolt képfájlt.

Rendeléskezelés (Orders)

Az Orders oldalon az összes beérkező rendelés látható, legújabbtól a legrégebbiig rendezve. Minden rendelési kártyán megjelenik a megrendelt termékek listája, a végösszeg, a szállítási adatok (vevő neve, osztálya) és a 4 jegyű átvételi kód. A búfés alkalmazott egy legördülő listából kiválaszthatja a rendelés aktuális státuszát (Készítés alatt, Elkészült, Átvéve), amelyet a rendszer azonnal elment a /api/order/status végponton. A kereső mező az átvételi kódra szűr: begépelve a diák által bemutatott kódot, azonnal megjelenik a keresett rendelés.

2.15 Hibakezelés és visszajelzések a felhasználónak

A GépészBüfé következetes hibaközlési stratégiát alkalmaz mind a backend, mind a frontend oldalon, hogy a felhasználó minden esetben értelmes visszajelzést kapjon.

Backend hibakezelés

Minden controller try-catch blokkal van körülvéve. Sikertelen művelet esetén a szerver { success: false, message: "..." } formátumú JSON-t küld vissza. A validator.isEmail() által érzékelt formátumsértés, a bcrypt.compare() által jelzett helytelen jelszó, a Mongoose duplicate key error (E11000) mind külön hibaüzenetekkel kerülnek lekeztelésre. A JWT middleware érvénytelen vagy lejárt token esetén 401 státuszkódot küld vissza, amelyre a frontend automatikusan törli a tárolt tokent és az Axios interceptor a bejelentkezési oldalra irányít.

Frontend visszajelzések

A felhasználói visszajelzések a react-toastify könyvtár toast értesítéseivel valósulnak meg. Sikeres bejelentkezéskor, regisztrációkor és rendelésleadáskor zöld toast jelenik meg, hiba esetén piros. Bejelentkezés nélküli rendelésleadási kísérletnél a rendszer a bejelentkezési modal-t nyitja meg, és nem engedi továbbnavigálni a felhasználót. Üres kosár esetén a rendelés leadása gomb disabled állapotban van, és egy figyelmezető üzenet jelenik meg. Az admin panel státuszváltásait szintén toast értesítés visszajelzi.

3. Összefoglalás és köszönetnyilvánítás

3.1 Összefoglalás

A projekt során sikeresen megvalósítottuk a kitűzött célokat: elkészítettük a GépészBüfé teljes értékű, háromrétegű webalkalmazást, amely lehetővé teszi az előzetes online rendelést, a rendelések valós idejű adminisztrátori kezelését és a 4 jegyű átvételi kódos azonosítást.

A munka során a legnagyobb technikai kihívást a kosár szinkronizálásának megoldása jelentette: biztosítani kellett, hogy a frontend lokális állapota és a MongoDB-ben tárolt adatok mindig konzisztensek maradjanak, akár bejelentkezés előtt, akár után módosít a felhasználó a kosarán. Ezt a StoreContext useEffect hook-jaival és feltételes API-hívásokkal oldottuk meg.

A projekt eredménye egy valóban használható, biztonságos és reszponzív alkalmazás, amelyet az iskola büféjének átadtunk tesztüzemeltetésre.

A fejlesztés egyben komoly tanulási folyamat is volt: mélyebben megismertük a JWT alapú hitelesítést, a Mongoose sémákat és a React Context API teljesítménytudatos alkalmazását. Ezeket a tapasztalatokat a továbbiakban szoftverfejlesztői munkánkban is hasznosítani kívánjuk.

3.2 Köszönetnyilvánítás

Köszönetünket szeretnénk kifejezni Pálfi Máté Gergely tanár úrnak, aki konzulensként szakmai iránymutatást és folyamatos visszajelzést nyújtott a projekt teljes időtartama alatt.

Köszönjük Jágri Ilona tanárnőnek a tesztelésben nyújtott segítséget és az értékes felhasználói visszajelzéseket, amelyek számos fejlesztési irányra hívták fel a figyelmet.

Végül köszönjük osztálytársainknak és barátainknak a türelmét, amellyel a fejlesztési időszak alatti fokozott elfoglaltságunkat elviselték.

4. Irodalomjegyzék

4.1 Médiaforrások

1. **React - https://www.youtube.com/watch?v=TtPXvEcE11E**
2. **MongoDB - https://www.youtube.com/watch?v=c2M-rlkkT5o**
3. **Express.js - https://www.youtube.com/watch?v=SccSCuHhOw0**
4. **Mern - https://www.youtube.com/watch?v=F9gB5b4jgOI**

4.2 Online források

5. React.js – Hivatalos dokumentáció https://react.dev/learn
6. Express.js – API Reference https://expressjs.com/en/4x/api.html
7. Mongoose ODM – Getting Started https://mongoosejs.com/docs/guide.html
8. JWT.io – Introduction to JSON Web Tokens https://jwt.io/introduction
9. MongoDB Atlas – Dokumentáció https://www.mongodb.com/docs/atlas/
10. Vite – Guide https://vitejs.dev/guide/
11. bcrypt npm csomag – Dokumentáció https://www.npmjs.com/package/bcrypt
12. MDN Web Docs – Fetch API https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API